Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет Информационных технологий и программирования

Лабораторная работа № 3

Ролевая модель в PostgreSQL

Выполнил:

Ягодин Захар Сергеевич, M3401

Санкт-Петербург

2026

Цель работы:

ознакомиться с концепцией ролей для управления разрешениями на доступ к объектам СУБД PostgreSQL. Получить навыки разграничения доступа.

```
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab3> cat .\docker-compose.yml
services:
  db:
    image: postgres:latest
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_USER: hutao
      POSTGRES_PASSWORD: hutao
      POSTGRES_DB: hutao
    ports:
      - "5432:5432"
```

```
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab3> docker compose up -d
[+] Running 2/2
 ✓ Network lab3_default   Created
 ✓ Container postgres_db  Started
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab3> docker exec -it postgres_db psql -U hutao -d hutao
psql (18.2 (Debian 18.2-1.pgdg13+1))
Type "help" for help.

hutao=#
```

Создание необходимых ролей

1. Создайте пять ролей:

- Роль, которая будет представлять вас. Имя роли — ваши имя и фамилия (например, ivan_ivanov).
- Групповые роли: itmo_users и itmo_readers - должны обладать правами на вход, создание баз данных и других ролей.
- Пользовательские роли: itmo_user_1 и itmo_reader_1 - роли должны обладать разрешением на вход.

```
hutao=# CREATE ROLE zahar_yagodin WITH LOGIN PASSWORD 'HUTAO' CREATEDB CREATEROLE;
CREATE ROLE
hutao=# CREATE ROLE itmo_readers WITH NOLOGIN NOCREATEDB NOCREATEROLE;
CREATE ROLE
hutao=# CREATE ROLE itmo_users WITH NOLOGIN NOCREATEDB NOCREATEROLE;
CREATE ROLE
hutao=# CREATE ROLE itmo_user_1 WITH LOGIN PASSWORD 'u1pwd' INHERIT;
CREATE ROLE
hutao=# CREATE ROLE itmo_reader_1 WITH LOGIN PASSWORD 'r1pwd' INHERIT;
CREATE ROLE
hutao=#
```

2. Добавьте:

- itmo_user_1 в itmo_users.
- itmo_reader_1 в itmo_readers.
- Пользовательские роли должны наследовать разрешения групповых ролей.

```
hutao=# GRANT itmo_users TO itmo_user_1;
GRANT ROLE
hutao=# GRANT itmo_readers TO itmo_reader_1;
GRANT ROLE
hutao=#
```

Создание базы данных

1. Создайте новую базу данных. В качестве владельца укажите Вашу роль.

```
GRANT ROLE
hutao=# CREATE DATABASE test OWNER zahar_yagodin;
CREATE DATABASE
hutao=#
hutao=# \c test zahar_yagodin;
You are now connected to database "test" as user "zahar_yagodin".
```

2. Подключитесь к новой базе данных и создайте 2-3 таблицы. Наполните их тестовыми значениями.

```
test=> CREATE TABLE games (
  id SERIAL PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  genre VARCHAR(50),
  rating INTEGER CHECK (rating >= 0 AND rating <= 10),
  price INTEGER
);
CREATE TABLE
test=> CREATE TABLE studios (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  country VARCHAR(50),
  ai_used BOOLEAN
);
CREATE TABLE
test=> INSERT INTO games (title, genre, rating, price) VALUES
('Metro 2033 Redux', 'FPS', 9, 700),
('Satisfactory', 'Automatization', 10, 9300),
('Stellar Blade', 'Action', 8, 27999),
('League of legends', 'MOBA', 7, 0);
INSERT 0 4
test=> INSERT INTO studios (name, country, ai_used) VALUES
('4A Games', 'Malta', false),
('Coffee Stain', 'Sweden', false),
('Shift Up', 'South Korea', true),
('Riot Games', 'USA', true);
INSERT 0 4
test=>
```

```
test=> select * from games;
 id |       title        |      genre      | rating | price
----+--------------------+-----------------+--------+-------
  1 | Metro 2033 Redux   | FPS             |      9 |   700
  2 | Satisfactory       | Automatization  |     10 |  9300
  3 | Stellar Blade      | Action          |      8 | 27999
  4 | League of legends  | MOBA            |      7 |     0
(4 rows)

test=> select * from studios;
 id |     name     |   country   | ai_used
----+--------------+-------------+---------
  1 | 4A Games     | Malta       | f
  2 | Coffee Stain | Sweden      | f
  3 | Shift Up     | South Korea | t
  4 | Riot Games   | USA         | t
(4 rows)
```

Установка привилегий

1. От имени своей роли:

- Выдайте itmo_users разрешения на SELECT, INSERT, UPDATE, DELETE во всех таблицах базы данных.
- Выдайте itmo_readers только разрешение на SELECT.

```
test=> GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO itmo_users;
GRANT
test=> GRANT SELECT ON ALL TABLES IN SCHEMA public TO itmo_readers;
GRANT
test=>
```

```
test=> GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO itmo_readers;
GRANT
test=> GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO itmo_users;
GRANT
```

Проверка прав

1. Подключитесь к БД под itmo_user_1:

- Выполните чтение записей.
- Попробуйте вставить новую запись.
- Проверьте возможность создания новой таблицы.

```
test=> \conninfo
        Connection Information
      Parameter       |        Value
----------------------+--------------------
 Database             | test
 Client User          | itmo_user_1
 Socket Directory     | /var/run/postgresql
 Server Port          | 5432
 Options              |
 Protocol Version     | 3.0
 Password Used        | false
 GSSAPI Authenticated | false
 Backend PID          | 222
 SSL Connection       | false
 Superuser            | off
 Hot Standby          | off
(12 rows)

test=> select * from games;
 id |       title        |     genre      | rating | price
----+--------------------+----------------+--------+-------
  1 | Metro 2033 Redux   | FPS            |      9 |   700
  2 | Satisfactory       | Automatization |     10 |  9300
  3 | Stellar Blade      | Action         |      8 | 27999
  4 | League of legends  | MOBA           |      7 |     0
(4 rows)

test=> select * from studios;
 id |     name     |   country   | ai_used
----+--------------+-------------+---------
  1 | 4A Games     | Malta       | f
  2 | Coffee Stain | Sweden      | f
  5 | Shift Up     | South Korea | t
  6 | Riot Games   | USA         | t
(4 rows)

test=> DELETE FROM studios WHERE ai_used=true;
DELETE 2
test=> select * from studios;
 id |     name     | country | ai_used
----+--------------+---------+---------
  1 | 4A Games     | Malta   | f
  2 | Coffee Stain | Sweden  | f
(2 rows)

test=> INSERT INTO studios (name, country, ai_used) VALUES
('Shift Up', 'South Korea', true),
('Riot Games', 'USA', true);
INSERT 0 2
test=> select * from studios;
 id |     name     |   country   | ai_used
----+--------------+-------------+---------
  1 | 4A Games     | Malta       | f
  2 | Coffee Stain | Sweden      | f
  7 | Shift Up     | South Korea | t
  8 | Riot Games   | USA         | t
(4 rows)

test=> CREATE TABLE testtable (
test(>   id SERIAL PRIMARY KEY,
test(>   name VARCHAR(50) NOT NULL
test(> );
ERROR:  permission denied for schema public
LINE 1: CREATE TABLE testtable (
                    ^
test=>
```

2. Повторите действия для itmo_reader_1.

```
test=> DELETE FROM studios WHERE ai_used=true;
ERROR:  permission denied for table studios
test=> INSERT INTO studios (name, country, ai_used) VALUES ('test', 'test', true);
ERROR:  permission denied for table studios
test=> CREATE TABLE testtable (id SERIAL PRIMARY KEY, name VARCHAR(50) NOT NULL);
ERROR:  permission denied for schema public
LINE 1: CREATE TABLE testtable (id SERIAL PRIMARY KEY, name VARCHAR(...
                    ^
test=>
```

Дополнительные вопросы

1. Кто такой владелец БД и какими правами он обладает? В чем его отличие от суперпользователя?
   Владелец БД (Owner) — это роль, которой назначено владение конкретной базой данных.
   - **Права:** Владелец обладает полными правами на все объекты внутри своей БД (таблицы, схемы, индексы), может изменять параметры базы и удалять её.
   - **Отличие от суперпользователя:** В то время как полномочия владельца ограничены рамками конкретной БД, суперпользователь обладает неограниченными правами во всей СУБД. Он игнорирует любые проверки прав доступа, может управлять системными файлами и изменять глобальные настройки сервера.
2. Какие механизмы безопасности можно использовать в PostgreSQL для предотвращения несанкционированного доступа?
   - **Аутентификация на уровне хоста:** файл конфигурации `pg_hba.conf` регламентирует, какие пользователи могут подключаться к серверу с определенных IP-адресов.
   - **Шифрование паролей**
   - **Row-Level Security (RLS):** механизм защиты на уровне строк, позволяющий ограничить видимость данных в таблице в зависимости от текущей роли пользователя.
3. Как можно организовать многоуровневый контроль доступа в базе данных?
   Многоуровневый контроль реализуется через иерархию разрешений (Privilege Hierarchy):
   1. **Уровень подключения:** Право `CONNECT` к базе данных.
   2. **Уровень схемы:** Право `USAGE` на использование конкретной схемы (например, `public`).
   3. **Уровень объектов:** Права `SELECT`, `INSERT`, `UPDATE`, `DELETE` на конкретные таблицы или представления.
   4. **Уровень столбцов:** Назначение прав доступа к отдельным атрибутам (колонкам) таблицы.
4. Какие преимущества у ролевой модели в PostgreSQL перед традиционной моделью пользователей?
   - **Масштабируемость:** Права назначаются не индивидуально, а группам ролей (например, `itmo_readers`). Новым пользователям достаточно выдать членство в этой роли.
   - **Удобство администрирования:** Изменение прав одной роли автоматически обновляет доступ для всех её членов (наследование прав через `INHERIT`).
   - **Безопасность:** Минимизируется риск ошибок при ручной настройке доступа для большого количества сотрудников.
5. Как можно защитить PostgreSQL от атак методом подбора пароля (brute-force)?
   - `auth_delay` - использование модуля, добавляющего задержку перед ответом сервера после неудачной попытки входа.
   - **Fail2Ban**
   - **SSL-сертификаты** - использование аутентификации по клиентским сертификатам вместо классических паролей.
6. Какие меры безопасности нужно учитывать при передаче конфиденциальных данных между базами?
   - **Шифрование трафика (SSL/TLS)** - гарантирует защиту данных от перехвата в процессе передачи по сети.
   - **Принцип наименьших привилегий** - технический пользователь, осуществляющий связь между БД, должен обладать минимально необходимым набором прав для выполнения конкретной транзакции.